

Credit Card Fraud Detection

Group 22

Team Member	Student ID	Contributions
Elangovan Sushmitha	1007050	- Code for main model - Code to reproduce graphs without training the model from scratch
V Priyanka Valli	1007021	- Completed report - Completed slides - Trained a model we did not proceed with - Worked on the code for the comparison models (Logistic Regression, Autoencoder)

Github Link to project: <https://github.com/esushmitha/Deep-Learning-Project-2026>

Introduction and Objectives

Credit card fraud is an anomaly-detection problem. The vast majority of transactions are legitimate, while only a small fraction are fraudulent. This severe class imbalance makes standard classification pipelines insufficient. Thus, we have developed a model with the primary goal of accurately distinguishing between legitimate (non-fraud) and fraudulent transactions. This project builds, trains, and evaluates a deep feedforward neural network in PyTorch to flag fraudulent transactions.

Dataset

Source: Kaggle - [Credit card Fraud data](#)

The dataset contains simulated credit-card transaction records with the binary target is_fraud (1 = fraudulent, 0 = legitimate). Raw features include transaction amount, merchant category, cardholder demographics, geolocation coordinates, and timestamps.

Property	Value
Raw file name	fraudTest.csv
Total samples	555719
Fraud Rate (class imbalance)	0.386%
Input features (after feature engineering)	22
Inputs	Merchant, category, amt, gender, city, state,

	zip, lat, long, city_pop, job, unix_time, merch_lat, merch_long, trans_hour, trans_day, trans_month, trans_dayofweek, is_weekend, age, distance, amt_log
Target variable	Is_fraud (binary: 0 / 1) A single sigmoid-activated scalar representing the predicted probability that the transaction is fraudulent. A decision threshold of 0.30 is applied to produce the binary label.

Preprocessing pipeline - preprocess.py

All preprocessing is implemented in src/preprocess.py. The pipeline is strictly fit on the training split only. Validation and test splits are transformed using the saved files (scaler, label encoders) to prevent data leakage.

	Steps Taken	Description
1	Load the raw data	Read fraudTest.csv, inspect shape and any missing values.
2	Drop missing targets	Rows with null is_fraud are removed.
3	Feature engineering	Extracts - trans_hour, trans_day, trans_month, trans_dayofweek, is_weekend → unusual transaction times - Age → demographic differences - Distance → geographic anomalies - Amt_log → stabilises skewed distribution
4	Drop leakage columns	Remove trans_num, cc_num, first, last, street, raw timestamps.
5	Fill missing values	Numeric: median imputation. Categorical: fill with 'Unknown'.
6	Train / Validation / Test split	80% / 10% / 10% (stratified)
7	Label encoding	Fit LabelEncoder per categorical column on train set, apply to validation and test sets.
8	Standard scaling	Fit StandardScaler on train features; transform validation and test.
9	Saved files	scaler.pkl, label_encoders.pkl, feature_columns.pkl saved to models/

Model Architecture - FraudNet

The model is a deep feedforward neural network implemented as FraudNet in `src/train.py`. It is replicated in `evaluate.py` and `test.py` for loading. It follows a progressively narrowing structure from 256 to 1 neuron, with Batch Normalisation, ReLU activations, and Dropout at each hidden layer for regularisation.

Layer	Output Dimensions	Activation	BatchNorm	Dropout
Input	Input Dimensions	-	-	-
Layer 1	256	ReLU	Yes	0.35
Layer 2	128	ReLU	Yes	0.30
Layer 3	64	ReLU	Yes	0.20
Layer 4	32	ReLU	No	0.10
Output	1	Sigmoid	No	-

Weights are initialised through the Kaiming Normal (He initialisation) for all Linear layers, biases are set to zero. This is appropriate for ReLU networks. Sigmoid is applied via `torch.sigmoid()` at inference time, not inside the model. During training, raw logits are passed to FocalLoss which applies `binary_cross_entropy_with_logits` internally for numerical stability.

Training procedure - train.py

To handle the class imbalance, two complementary strategies are used:

1. Focal Loss function
Focal Loss down-weights easy negatives (the many non-fraud samples) and focuses training on hard, misclassified examples.
2. Class-weight computation
The sklearn's `compute_class_weight('balanced')` is used to compute a fraud up-weight, capped at 10.0 to prevent excessive false-positive sensitivity.

Hyperparameters used	Value
Optimizer	Adam
Learning Rate	$3e^{-4}$
Weight Decay (L2 Regularisation)	$1e^{-4}$
Batch Size	1024
Maximum Epochs	50
Early Stopping Patience	8 epochs (monitors validation loss)
Learning Rate Scheduler	ReduceLROnPlateau (factor=0.5, patience=3, min_lr=1e-6)

Focal Loss gamma	2.0
Focal Loss alpha	0.25
Random Seed	42

After training on the above hyperparameters, the best decision threshold was selected on the validation set by maximising the F1-score across the full precision-recall curve. The chosen threshold (0.30) was saved to models/threshold.json and was applied unchanged at test time, to ensure no leakage from the test set.

The model state-dict is saved with torch.save() to models/is_fraud_model.pth whenever validation loss improves.

Evaluation Results - evaluate.py

All metrics below are computed on the held-out test set (10% of data, never seen during training or threshold selection) at threshold = 0.30.

In addition to the standard classification metrics (Accuracy, Precision, Recall, and F1-Score), we also used two threshold-independent metrics: ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) and PR-AUC (Precision-Recall - Area Under the Curve).

ROC-AUC measures the ability of the model to discriminate between classes across all possible decision thresholds. At each threshold, it plots the True Positive Rate (Recall) against the False Positive Rate, and the AUC summarises this curve as a single number between 0 and 1, where 1 represents perfect discrimination.

PR-AUC similarly sweeps across all thresholds, but instead plots Precision against Recall. A key property of PR-AUC is that it focuses entirely on the positive (fraud) class. It is not influenced by the large number of true negatives that dominate an imbalanced dataset. A random classifier on this dataset would achieve a PR-AUC approximately equal to the fraud rate, making any score significantly above that a meaningful indicator of real detection ability.

While ROC-AUC is included as a secondary metric for completeness and comparability with other published work, PR-AUC is our primary evaluation metric for this project. This is because our dataset is severely class-imbalanced, and ROC-AUC can be misleadingly optimistic in such settings. The vast number of true negatives artificially deflates the False Positive Rate, making the ROC curve appear better than the model's practical fraud-catching ability warrants. PR-AUC does not suffer from this problem, and therefore gives a more honest and meaningful reflection of how well the model detects the rare fraud cases that matter most in a real-world deployment.

Metric	Values	Comments
Accuracy	99.84%	Not the primary metric used in evaluation. It is just high due to class imbalance. $\frac{55323 (TN) + 159 (TP)}{55323 (TN) + 159 (TP) + 34 (FP) + 56 (FN)} \times 100\% = 99.84\%$

Precision	82.38%	Of the flagged transactions, 82.38% are genuinely fraudulent. $\frac{159 (TP)}{159 (TP) + 34 (FP)} \times 100\% = 82.38\%$
Recall	73.95%	Of all fraudulent transactions, the model caught 73.95% of them. $\frac{159 (TP)}{159 (TP) + 56 (FN)} \times 100\% = 73.95\%$
F1-Score	77.94%	$2 \times \frac{82.38 \times 73.95}{82.38 + 73.95} = 77.94\%$
ROC-AUC	99.23%	Excellent discrimination across all thresholds
PR-AUC	81.96%	Strong performance on the imbalanced positive class

Confusion Matrix

	Predicted: Not Fraud	Predicted: Fraud
Actual: Not Fraud	55323 (TN)	34 (FP)
Actual: Fraud	56 (FN)	159 (TP)

Test Cases

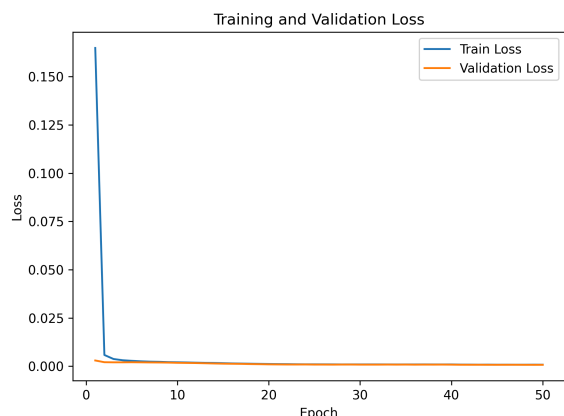
TOP 5 FALSE POSITIVES (Wrongly Flagged as Fraud)							
row_id	actual_label_name	predicted_label_name	predicted_probability	prediction_confidence	confidence_label	outcome	
24912	Not Fraud	Fraud	0.640055	0.640055	Moderate	FALSE	POSITIVE
45042	Not Fraud	Fraud	0.568496	0.568496	Low	FALSE	POSITIVE
2356	Not Fraud	Fraud	0.544642	0.544642	Low	FALSE	POSITIVE
49486	Not Fraud	Fraud	0.530115	0.530115	Low	FALSE	POSITIVE
47081	Not Fraud	Fraud	0.500166	0.500166	Low	FALSE	POSITIVE

TOP 5 FALSE NEGATIVES (Missed Fraud Cases)							
row_id	actual_label_name	predicted_label_name	predicted_probability	prediction_confidence	confidence_label	outcome	
13175	Fraud	Not Fraud	0.025635	0.974365	Very High	FALSE	NEGATIVE
20762	Fraud	Not Fraud	0.031660	0.968340	Very High	FALSE	NEGATIVE
23331	Fraud	Not Fraud	0.039598	0.960402	Very High	FALSE	NEGATIVE
40612	Fraud	Not Fraud	0.045646	0.954354	Very High	FALSE	NEGATIVE
35184	Fraud	Not Fraud	0.055559	0.944441	Very High	FALSE	NEGATIVE

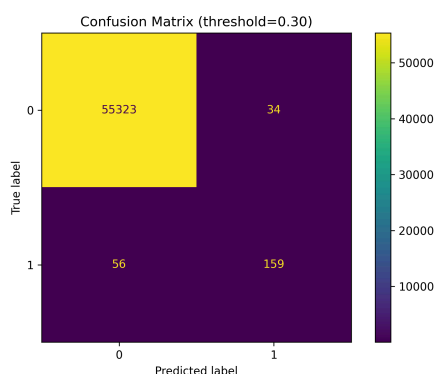
Visualisations - plot_evaluation_results.py

All figures are generated by src/plot_evaluation_results.py. To reproduce any figure, run: python src/plot_evaluation_results.py from the project root. Figures are saved to output/evaluation_plots/.

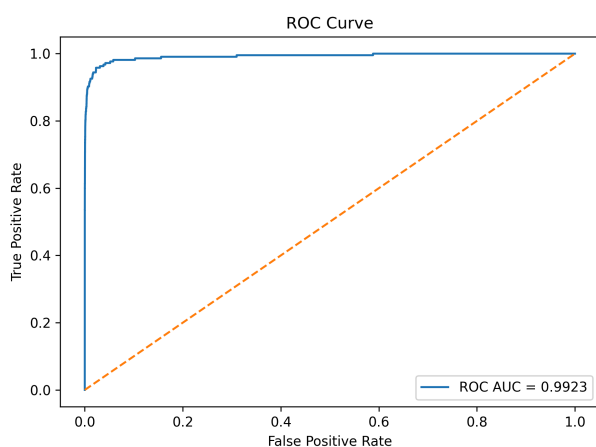
Figure	Comments
--------	----------



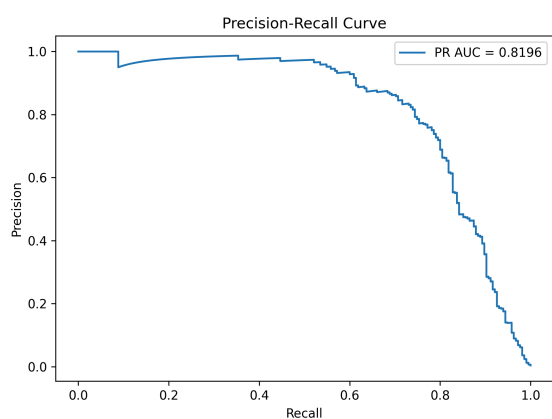
This figure shows the Focal Loss for both the training and validation sets across 50 epochs. The training loss starts high and drops sharply within the first 2 to 3 epochs, after which both curves stabilise and converge closely together near zero. The validation loss remains consistently low throughout training and never diverges from the training loss, indicating that the model is not overfitting. The smooth convergence suggests that the regularisation techniques applied were effective.



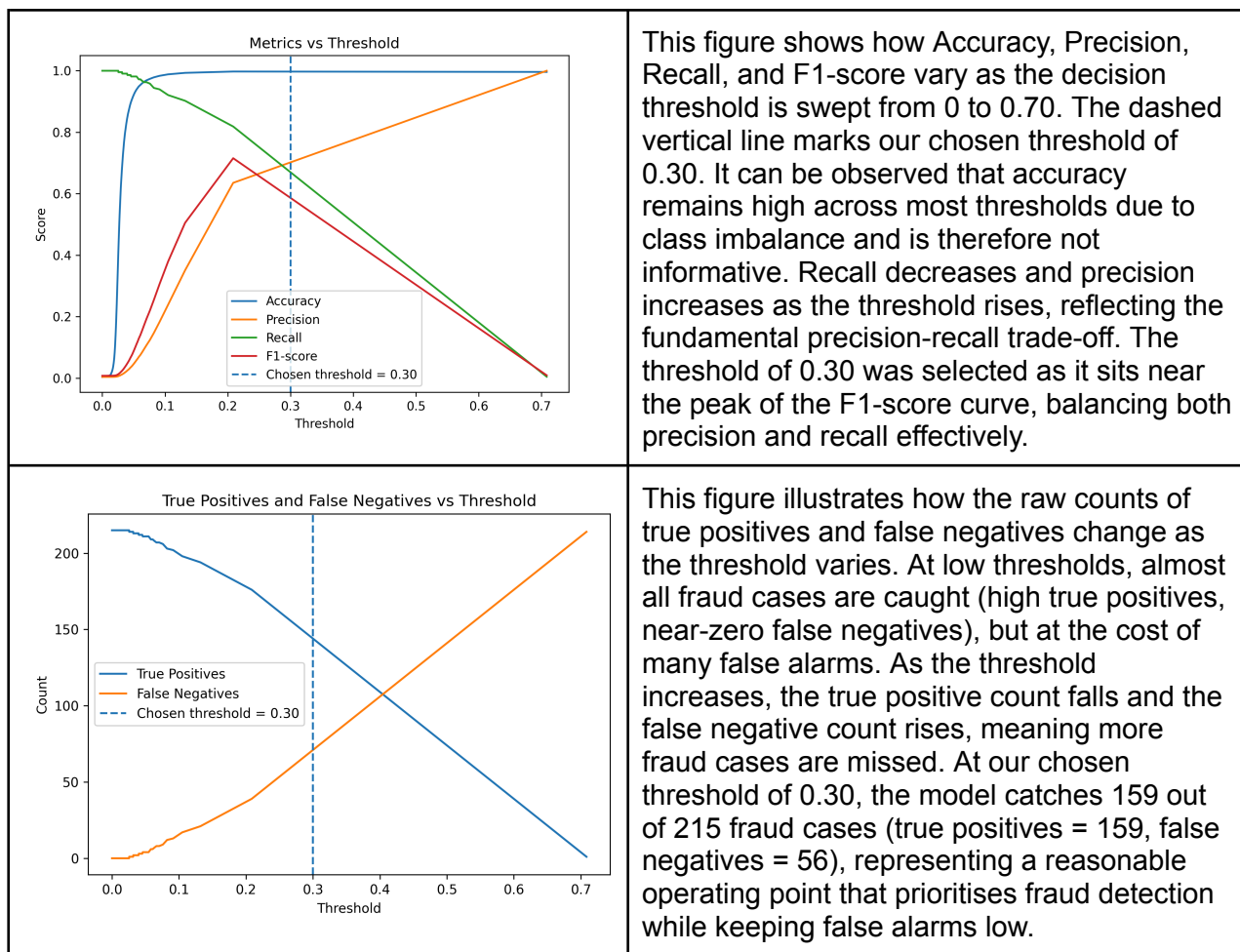
The confusion matrix shows the model's predictions on the test set at the chosen decision threshold of 0.30. Out of 55,357 legitimate transactions, 55,323 were correctly classified as non-fraud (true negatives) and only 34 were incorrectly flagged as fraud (false positives). Out of 215 actual fraud cases, 159 were correctly detected (true positives) and 56 were missed (false negatives).



The ROC curve plots the True Positive Rate against the False Positive Rate across all possible decision thresholds. The curve rises steeply toward the top-left corner, indicating that the model achieves high recall with very few false alarms across a wide range of thresholds. The area under the curve of 0.9923 confirms excellent overall discrimination ability between fraudulent and legitimate transactions.



The Precision-Recall curve shows the trade-off between precision and recall across all thresholds. The curve remains at near-perfect precision for low-to-moderate recall values, before declining as the model is pushed to catch more fraud cases. A PR-AUC of 0.8196 is strong for a dataset with such severe class imbalance, where a random classifier would score approximately 0.0058 (equal to the fraud rate). This is our primary evaluation metric.



Discussion

The model achieves a ROC-AUC of 99.23%, indicating near-perfect discrimination ability across all thresholds. At the chosen threshold of 0.30, it catches 73.95% of fraud cases with a false-positive count of only 34 out of 55,357 legitimate transactions which is a very low alarm rate in practice. However, the remaining 26% of missed fraud (56 false negatives) is the primary concern. Inspection of the highest-confidence false negatives shows that many are low-value transactions in merchant categories that overlap heavily with normal spending patterns. Potential reasons include:

- Distributional overlap: some fraudulent transactions are indistinguishable from legitimate ones on the available features alone.
- Limited temporal features: the model does not capture sequential patterns (e.g. multiple small charges in quick succession)
- Synthetic dataset limitations: the Kaggle dataset is simulated; real-world fraud patterns may be more complex.

To contextualise the performance of FraudNet, we trained and evaluated two baseline models on the exact same train/validation/test splits and preprocessing pipeline.

Logistic Regression is a linear classifier that models the log-odds of fraud as a linear combination of the input features. It serves as a standard shallow baseline for binary classification tasks.

Autoencoder is an unsupervised anomaly detection approach where the model is trained to reconstruct normal (non-fraud) transactions. At inference time, transactions with high reconstruction error are flagged as anomalies (fraud). This is a natural baseline for anomaly detection problems since it does not rely on fraud labels during training.

Model	ROC-AUC	F1 (fraud)	Recall (fraud)
FraudNet (our project)	0.9923	0.7794	0.7395
Logistic Regression	0.9614	0.07	0.86
Autoencoder	0.8651	0.18	0.33

The results reveal an important distinction between ROC-AUC and practical detection ability, which reinforces our choice of PR-AUC and F1 as primary metrics. Logistic Regression achieves a high ROC-AUC of 0.9614 and a high recall of 0.86, but its F1-score of only 0.07 exposes a critical weakness. It flags an enormous number of legitimate transactions as fraud (very low precision), making it impractical in deployment despite catching many real fraud cases. This is a classic symptom of a model that has simply learned to predict fraud aggressively to compensate for class imbalance, without genuinely discriminating between the two classes.

The Autoencoder performs poorly across all three metrics. With a recall of only 0.33, it misses the vast majority of fraud cases entirely, suggesting that the reconstruction-error approach struggles on this dataset because fraudulent transactions are not sufficiently distinct from legitimate ones in the feature space for the model to separate them at inference time.

FraudNet achieves the best balance across all metrics. Its F1-score of 0.7794 is dramatically higher than both baselines, demonstrating that it has learned to genuinely discriminate between fraud and non-fraud rather than simply exploiting the class imbalance. The ROC-AUC of 0.9923 further confirms its strong overall discrimination ability. These results validate the design choices made in FraudNet specifically, the use of Focal Loss and threshold tuning on the validation set as effective strategies for handling severe class imbalance in a real-world anomaly detection setting.

Reproducibility and steps to run the code

Package dependencies

Package	Version (recommended)
Python	3.9+
torch	2.0+
numpy	1.24+
pandas	2.0+
scikit-learn	1.3+
joblib	1.3+

matplotlib	3.7+
------------	------

Step-by-step instructions to reproduce from scratch

1. Clone the repository. Download dataset and place fraudTest.csv in data/raw/.
2. Install dependencies: `pip install torch numpy pandas scikit-learn joblib matplotlib`
3. Run preprocessing: `python src/preprocess.py` — Produces train/val/test CSVs and saves `scaler.pkl`, `label_encoders.pkl`, `feature_columns.pkl`
4. Train the model: `python src/train.py` — Saves `is_fraud_model.pth` and `threshold.json` to `models/`
5. Evaluate on test set: `python src/evaluate.py` — Prints metrics and saves `test_metrics.json` and `test_predictions.csv`
6. Generate all figures: `python src/plot_evaluation_results.py` — Saves 8 PNG plots to `output/evaluation_plots/`
7. Inspect predictions: `python src/test.py` — Prints top-5 TP/FP/FN/TN cases; saves full results to `output/`

Loading the saved model to reproduce the graphs without retraining:

```
python src/evaluate.py
python src/plot_evaluation_results.py
```